

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Языки программирования»
Вариант 15

Работу выполнил
студент группы ИТ-7,8-2025 1 курса
Рогозин Е.А.
«14» июня 2026 г.

Работу проверил
Шилина А.В.
«15» июня 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

Постановка задачи №1	3
Алгоритм решения задачи №1	3
Тестирование задачи №1	7
Код программы №1	8
Постановка задачи №2	11
Алгоритм решения задачи №2	11
Тестирование задачи №2	13
Код программы №2	16
Ссылка на репозиторий Github	19

Постановка задачи №1

Задача 15

Используя структуру стека подсчитать значение арифметического выражения, записанного в обратной польской записи (постфиксной записи) при условии, что используются знаки операций $+$, $-$, $*$, $/$ и операнды являются вещественными положительными числами. Во вводимой пользователем строке числа и знаки операций разделяются одним пробелом.

Замечание: Постфиксной формой записи выражения $a\ b$ называется запись, в которой знак операции размещен за операндами $a\ b$.

Примеры:

$$a - b \rightarrow a\ b\ -$$

$$a * b + c \rightarrow a\ b\ * \ c\ +$$

$$a * (b + c) \rightarrow a\ b\ c\ +\ *$$

$$a + b / c / d * e \rightarrow a\ b\ c\ / \ d\ / \ e\ * \ +$$

Алгоритм решения задачи №1

Для решения задачи вычисления арифметического выражения в обратной польской записи ключевой структурой данных является стек, работающий по принципу LIFO (Last In, First Out — «последним пришел, первым ушел»).

Почему стек необходим в данной программе: В обратной польской записи знаки операций следуют после операндов. При чтении строки слева направо числа должны накапливаться в памяти до тех пор, пока не встретится знак операции. Операция всегда применяется к двум последним считанным числам. Стек идеально подходит для этого, так как предоставляет мгновенный доступ к последним считанным данным.

Реализация на языке Python: В качестве реализации стека выбран список (list) — встроенный динамический массив.

Метод “`.append()`” выполняет операцию добавления элемента в конец стека.

Метод “`.pop()`” выполняет операцию извлечения последнего элемента.

Это обеспечивает максимальное быстродействие алгоритма по времени и не требует импорта сторонних библиотек.

Основной алгоритм вычисления выражения.

Алгоритм последовательно обрабатывает строку, разбитую на элементы (токены).

1. Если токен является числом, оно переводится в вещественный тип (float) и помещается в стек. Если число отрицательное, программа выдаст ошибку.
2. Если токен является оператором (+ - / *), из стека извлекаются два верхних числа. При этом первое извлеченное число становится правым операндом (b), а второе — левым (a). Над этими числами выполняется операция, и результат возвращается в стек.
3. Если токен не является ни числом, ни оператором, программа выдает ошибку ввода.
4. После обработки всех токенов в стеке должен остаться ровно один элемент — результат выражения, либо больше одного элемента, если при вводе выражения допущена ошибка.

Псевдокод

Функция `grp_evaluate(строка_выражения)`:

1. Разбить строку_выражения по пробелам на список элементов (токенов)

2. Создать пустой стек (список)

3. Если список_элементов пустой:

 Вернуть "Ошибка: Введена пустая строка."

4. Для каждого элемента из списка:

 А. ЕСЛИ элемент — это оператор ('+', '-', '*', '/');

 Если в стеке меньше 2 чисел:

 Вернуть "Ошибка: Недостаточно чисел для операции."

 Извлечь из стека последнее число и записать в В (правый операнд)

 Извлечь из стека текущее последнее число и записать в А (левый операнд)

 Если оператор '+', то Результат = $A + B$

 Если оператор '-', то Результат = $A - B$

 Если оператор '*', то Результат = $A * B$

 Если оператор '/', то Результат = A / B

 Положить Результат обратно на вершину стека

 Б. ИНАЧЕ (если элемент это не оператор):

 Попробовать перевести элемент в вещественное число.

 Если не получилось (если это не число):

 Вернуть "Ошибка: Некорректный символ."

 Если число меньше или равно 0:

 Вернуть "Ошибка: Числа должны быть положительными."

 Положить число на вершину стека

5. После проверки всех элементов:

 Если в стеке осталось больше одного числа:

 Лишние_числа = (Количество элементов в стеке) - 1

 Вернуть "Ошибка: В стеке остались лишние числа, лишних чисел = {лишние_числа}"

6. Вернуть первое (и единственное) число из стека как итоговый Ответ.

Конец Функции

Архитектура программы

Программа построена по функциональному принципу и представляет собой единый скрипт с четким разделением обязанностей между функциями:

Блок `__main__`: Отвечает за бесконечный цикл интерфейса, перехватывает пользовательские строки, определяет, команда это или выражение, и маршрутизирует управление.

Далее в блоке `__main__` в цикле интерфейса производится очистка ввода пользователя от лишних пробелов и преобразование заглавных букв в строчные с помощью встроенных функций `.strip()` и `.lower()`.

После преобразования строки идет проверка ввода и вызов функций которые запрашивает пользователь:

1. При вводе “help” выводится информация о командах без вызова функций.
2. При вводе “examples” вызывается вспомогательная функция `examples()` которая выводит образцы выражений и вызывает функцию `grp_evaluate()` для их решения.
3. При вводе “clear” вызывается вспомогательная функция `clear()` которая очищает консоль.
4. При вводе “exit” вызывается вспомогательная функция `exit()` для завершения программы.
5. При вводе выражения или несуществующей команды вызывается функция `grp_evaluate()` для решения введенного выражения.

Примечание: в обработке исключений продуман случай с вводом несуществующей команды.

Комплексная обработка ошибок

Программа предотвращает аварийное завершение процесса при любых некорректных действиях пользователя:

1. **Защита от синтаксического мусора (try, except):** При попытке ввести текст (например, "67 2 плюс") программа плавно обрабатывает исключение конвертации и сообщает, какой именно токен вызвал сбой.
2. **Проверка логики:** Строго соблюдается условие задачи. Отрицательные числа и нули отсекаются на этапе попадания в стек.
3. **Контроль структуры обратной польской записи:** Если знаков операций больше, чем нужно, программа предотвращает ошибку выхода за границы индекса (`IndexError`) и выводит понятное сообщение.
4. **Подсчёт избыточности вычислений:** Реализован математический анализ состояния стека после завершения разбора. Если пользователь забыл указать операторы (5 6 7 +), программа высчитывает точное количество лишних чисел.
5. **Примечание по делению на ноль:** Условием задачи жестко заданы только положительные вещественные числа, что исключает появление нуля в знаменателе.

Пользовательский интерфейс

- 1. Стойкость к ошибкам ввода:** Использование цепочки методов `.strip().lower()` гарантирует, что случайные пробелы по краям строки или ввод команд в верхнем регистре (к примеру HeLp) будут корректно интерпретированы программой.
- 2. Очистка консоли:** Предусмотрена печать 100 пустых строк как альтернатива очистки через библиотеки `os` и `subprocess`.
- 3. Интерактивный диалог:** После каждого вычисления программа предлагает сделать выбор (Y/N) для продолжения работы, предотвращая случайное закрытие окна с результатом.

Тестирование задачи №1

Для тестирования задачи в программе есть вспомогательная функция с примерами выражений. На рис.1 будет представлена работа программы с выражениями и результатами.

```
Вычисление выражений в обратной польской записи (RPN).  
Введите выражение в обратной польской записи или команду (Введите 'help' для вызова меню команд): examples  
Пример 1: 5 2 / Ответ: 2.5  
Пример 2: 5 10 2 / 2.5 / 4 * + Ответ: 13.0  
Пример 3: 5 -2 + Ошибка: Все числа должны быть положительными.  
Пример 4: 5 3 + * Ошибка: Недостаточно чисел в стеке для операции '*'.  
Пример 5: 5 3 2 + Ошибка: Выражение составлено неверно. В стеке остались лишние числа. Количество лишних чисел: 1  
Пример 6: 5 one + Ошибка: Некорректный символ, элемент или команда: 'one'.
```

Рис.1

В примерах собраны все случаи: правильная работа программы и всевозможные ошибки ввода.

Анализ примеров

- «5 2 /» = $5/2=2.5$ - корректный пример, программа посчитала его правильно.
- «5 10 2 / 2.5 / 4 * +» = $5+(10/2/2.5*4)=13$ - корректный пример, программа посчитала его правильно.

3. «5 -2 +» = $5+(-2)$ - число -2 не положительное, программа выдала ошибку.

4. «5 3 + *» - лишний знак, программа выдала ошибку.

5. «5 3 2 +» - не хватает знака, программа выдала ошибку.

6. «5 one +» - некорректный ввод, программа выдала ошибку.

Программа обработала все ошибки и выдала причину что не так в вводе.

Проверка работы программы с пользовательским вводом

```
Введите выражение в обратной польской записи или команду (Введите 'help' для вызова меню команд): 1 2 3 * +  
Ваше выражение: 1 2 3 * +  
Вывод: Ответ: 7.0  
Введите 'Y' чтобы ввести ещё одно выражение, либо любой другой символ для выхода: █
```

Рис.2

На рис.2 данные введены с клавиатуры, программа распознала ввод как выражение и правильно его посчитала ($1\ 2\ 3\ * + = 1 + (2 * 3) = 7$)

Код программы №1

```
def rpn_evaluate(expression):
    tokens = expression.split() # Чтение выражения
    stack = []

    # Exception: Если строка пустая
    if not tokens:
        return "Ошибка: Введена пустая строка."

    for token in tokens:
        if token in ('+', '-', '*', '/'):
            # Exception: В стеке меньше двух чисел
            if len(stack) < 2:
                return f"Ошибка: Недостаточно чисел в стеке для операции '{token}'."

            b = stack.pop() # LIFO: второй операнд будет последним в стеке
            a = stack.pop()

            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(a / b) # Проверка деления на 0 не нужна, так как все числа
положительные
        else:
            # Exception: Элемент не является вещественным числом
            try:
                number = float(token)
                # Exception: Число не является положительным
                # Exception: В вводе обнаружены символы которые не являются числами или
знаками операций
                if number <= 0:
                    return f"Ошибка: Все числа должны быть положительными."
                stack.append(number)
            except ValueError:
                return f"Ошибка: Некорректный символ, элемент или команда: '{token}'."

    # Exception: В вводе обнаружены лишние числа
    if len(stack) != 1:
        extra_count = len(stack) - 1
        return f"Ошибка: Выражение составлено неверно. В стеке остались лишние числа.
Количество лишних чисел: {extra_count}"

    return f"Ответ: {stack[0]}"

def examples(): # Вспомогательная функция: примеры ввода
    # 1. Корректный пример
```



```

print("Пример 1: 5 2 / ", rpn_evaluate("5 2 /"))

# 2. Корректный пример
print("Пример 2: 5 10 2 / 2.5 / 4 * + ", rpn_evaluate("5 10 2 / 2.5 / 4 * +"))

# 3. Ошибка: Число меньше нуля
print("Пример 3: 5 -2 + ", rpn_evaluate("5 -2 +"))

# 4. Ошибка: Лишний оператор
print("Пример 4: 5 3 + * ", rpn_evaluate("5 3 + *"))

# 5. Ошибка: Лишнее число
print("Пример 5: 5 3 2 + ", rpn_evaluate("5 3 2 +"))

# 6. Ошибка: Некорректный символ
print("Пример 6: 5 one + ", rpn_evaluate("5 one +"))

print("Примечание для ввода: Числа и знаки выражения разделяются пробелами.")

def clear(): # Вспомогательная функция: очистка консоли
    print("\n" * 100) # Вывод пустых строк
    print("Консоль очищена.")

def exit(): # Вспомогательная функция: Завершение программы
    print("Выход из программы...")
    return False

if __name__ == "__main__":

    print("Вычисление выражений в обратной польской записи (RPN).")

    work = True

    while work:
        expression = input("Введите выражение в обратной польской записи или команду (Введите
'help' для вызова меню команд): ")
        expression = expression.strip().lower() # Убирает лишние пробелы в начале и конце,
превращает заглавные буквы в строчные

        if expression == 'help':
            print("Командное меню:")
            print("'examples' - Показать примеры ввода и итогового результата после их решения
программой.")
            print("'clear' - Очистить консоль.")
            print("'exit' - Завершить программу.")
        elif expression == 'examples':
            examples()
        elif expression == 'exit':
            work = exit()
        elif expression == 'clear':
            clear()
        else:

```

```
        print(f"Ваше выражение: {expression}")
        print("Вывод:", rpn_evaluate(expression))
        ifcontinue = input("Введите 'Y' чтобы ввести ещё одно выражение, либо любой другой
символ для выхода: ")
        if ifcontinue.lower() != 'y':
            work = exit()
        print("Программа завершена.")
```

Постановка задачи №2

Задача 19

В файле даны n целых чисел, и здесь же указан путь их размещения в бинарном дереве виде двоичного кода (коды не повторяются). Построить двоичное дерево целых чисел, в котором путь по дереву определяется указанным двоичным кодом в этом листе (1 – переход к правому потомку, 0 – переход к левому потомку). В корень автоматически заносится значение 0.

Учитывать ситуацию, когда дерево не может быть построено.

Алгоритм решения задачи №2

Обоснование выбора структур данных

Для реализации поставленной задачи были выбраны следующие структуры данных:

Динамическая связанная структура (Узловое представление дерева):

В качестве основы используется класс `TreeNode`. Каждый узел содержит указатель на значение (`value`) и две ссылки на потомков (`left`, `right`).

Списки:

Массив пар `data_pairs` для хранения считанных промежуточных значений до момента компиляции дерева. Обеспечивает последовательный проход по набору входных данных за линейное время ($O(N)$).

Архитектура программы

Программа спроектирована по модульно-функциональному принципу с разделением зон ответственности.

1. Входные данные из файла или с клавиатуры (`read_from_file()`, `read_from_console()`).
2. Преобразование и валидация строк с данными. Разделение данных на валидные логические пары (`parse_line()`)
3. Основной алгоритм пошаговой сборки (`build_tree()`) и проверка на пустые промежуточные ячейки (без данных, с потомками) (`check_for_empty_nodes()`)
4. Основной алгоритм с интерфейсом, также отвечающий за запуск основного алгоритма построения (`main()`)

Описание ключевых алгоритмов

Программа начинает движение от корня для каждой пары (value, path). Для каждого символа пути ('0' или '1') проверяется существование потомка. Если его нет — создается пустой узел (если в ячейках 1 и 101 значения есть, а в 10 значения нет, программа выдаст ошибку). По завершении пути проверяется ячейка на предмет дублирования.

Псевдокод

```
Алгоритм СобратьДерево(список_данных):
    Создать Корень со значением 0
    Для каждой пары (Число, Путь) из список_данных:
        Текущий_Узел = Корень
        Для каждого Символа из Путь:
            Если Символ == "0" то:
                Если у Текущий_Узел нет левого потомка то:
                    Создать пустого левого потомка
                    Текущий_Узел = левый потомок
            Иначе Если Символ == "1" то:
                Если у Текущий_Узел нет правого потомка то:
                    Создать пустого правого потомка
                    Текущий_Узел = правый потомок
        # Проверка на перезапись данных
        Если у Текущий_Узел уже есть значение то:
            ОШИБКА: "Дерево построить нельзя, этот путь уже занят!"
        Остановить программу
        Записать Число в Текущий_Узел
    # Запуск проверки на пустые промежуточные ячейки
    ПроверитьУзлы(левый потомок Корня)
    ПроверитьУзлы(правый потомок Корня)
    Вернуть Корень

Алгоритм ПроверитьУзлы(Узел):
    Если Узел не существует то:
        Выйти из функции (вернуться назад)
    # Проверка на пустоту
    Если у Узел нет значения то:
        ОШИБКА: "Дерево разорвано! Найден пустой узел внутри структуры."
        Остановить программу
    # Рекурсивно идем проверять детей этого узла
    ПроверитьУзлы(левый потомок Узла)
    ПроверитьУзлы(правый потомок Узла)
```

Особенности реализации

Обработка исключений

Программа спроектирована с учетом концепции "Отказоустойчивость и безопасность ввода".

Ошибки разделены на два эшелона:

Ошибки окружения и синтаксиса:

При чтении данных сбои деления строк (если количество строк не равно двум) или ошибки преобразования типов (ValueError при вызове int()) локализуются внутри функции parse_line. Программа не прерывает работу аварийно, а пропускает поврежденную запись, логирует ошибку в консоль и переходит к обработке следующей строки.

Критические архитектурные ошибки (Аварийная остановка программы):

Если данные синтаксически верны, но противоречат законам построения дерева (Обнаружена перезапись данных или промежуточный узел без значения), генерируются явные исключения ValueError. Блок try-except в диспетчере main() перехватывает их, уничтожает ссылку на некорректно собранный граф и выводит пользователю развернутый вердикт (ошибку).

Пользовательский интерфейс

Гибкость ввода: Реализовано разветвление логики выполнения программы в зависимости от флага выбора. Интегрирована поддержка как автоматизированной обработки (пакетный режим через файлы конфигурации .txt), так и интерактивной (ручной ввод).

Маркер терминального состояния: Для ручного ввода применен паттерн Sentinel value — одиночный символ 0 сигнализирует системе о завершении ввода данных.

Тестирование программы №2

Тестирование проводилось с помощью пяти наборов исходных данных: 4 набора в файлах и 1 вручную.

Набор 1 (успешное построение дерева):

1 0
2 1
3 00
4 01

Результат работы - успешное построение дерева (рис.3).

```

Построение бинарного дерева по данным из файла.
Выберите способ ввода данных:
1: Прочитать данные из файла
2: Ввести вручную
Ваш выбор: 1
Введите имя или путь к файлу: set1.txt
Дерево успешно построено.

```

Рис.3

Набор 2 (ошибка синтаксиса):

5 0
два 10
67
7 11

Результат работы на рис.4. Программа выдала 3 ошибки: некорректное значение, некорректное количество элементов и существование промежуточной пустой ячейки.

```
Построение бинарного дерева по данным из файла.  
Выберите способ ввода данных:  
1: Прочитать данные из файла  
2: Ввести вручную  
Ваш выбор: 1  
Введите имя или путь к файлу: set2.txt  
Ошибка в записи в строке 2: 'два' не является целым числом  
Ошибка в записи в строке 3: ожидалось 2 элемента, получено 1  
  
Ошибка построения дерева Обнаружена пустая промежуточная ячейка по пути '1'. Дерево не может быть построено.
```

Рис.4

Набор 3 (перезапись значения):

5 01
6 01

Результат работы на рис.5. Программа нашла в ячейке «01» значение и не записала число 6 (вторую запись).

```
Построение бинарного дерева по данным из файла.  
Выберите способ ввода данных:  
1: Прочитать данные из файла  
2: Ввести вручную  
Ваш выбор: 1  
Введите имя или путь к файлу: set3.txt  
  
Ошибка построения дерева Обнаружена перезапись: путь '01' уже занят числом '5', невозможно записать число '6'.
```

Рис.5

Набор 4 (пустая промежуточная ячейка):

5 0
6 010

Результат работы на рис.6. Программа нашла промежуточную ячейку и выдала ошибку.

```
Построение бинарного дерева по данным из файла.  
Выберите способ ввода данных:  
1: Прочитать данные из файла  
2: Ввести вручную  
Ваш выбор: 1  
Введите имя или путь к файлу: set4.txt  
  
Ошибка построения дерева Обнаружена пустая промежуточная ячейка по пути '01'. Дерево не может быть построено.
```

Рис.6

Набор 5 (ввод с клавиатуры):

Входные данные и результат работы на рис.7.

```

Выберите способ ввода данных:
1: Прочитать данные из файла
2: Ввести вручную
Ваш выбор: 2

Выбран ручной ввод данных
Введите данные в формате: [число] [путь] через пробел
Для завершения ввода введите 0
-----
Запись 1: 1 0
Запись 2: 2 1
Запись 3: 3 10
Запись 4: 0
Ввод завершен.
Дерево успешно построено.

```

Рис.7

Программа обрабатывает ошибки так, как было задумано.

Другие проверки:

Проверка на некорректное название файла(рис.8)

```

Построение бинарного дерева по данным из файла.
Выберите способ ввода данных:
1: Прочитать данные из файла
2: Ввести вручную
Ваш выбор: 1
Введите имя или путь к файлу: iwannabetheguy.txt
Ошибка: Файл 'iwannabetheguy.txt' не найден.
Не найдены корректные данные для построения. Сборка дерева отменена.

```

Рис.8

Проверка на некорректный выбор типа ввода(рис.9, рис.10)

```

Построение бинарного дерева по данным из файла.
Выберите способ ввода данных:
1: Прочитать данные из файла
2: Ввести вручную
Ваш выбор: 3
Неверный выбор. Программа завершена.

```

Рис.9

```

Построение бинарного дерева по данным из файла.
Выберите способ ввода данных:
1: Прочитать данные из файла
2: Ввести вручную
Ваш выбор: hello
Неверный выбор. Программа завершена.

```

Рис.10

Код программы №2

```
class TreeNode:
    def __init__(self, value=None):
        self.value = value # Значение в узле
        self.left = None # Левый потомок (0)
        self.right = None # Правый потомок (1)

def build_tree(data_pairs):
    root = TreeNode(0) # В корень заносится значение 0

    # Построение и проверка на перезапись
    for value, path in data_pairs:
        current = root

        for char in path:
            if char == '0':
                if current.left is None:
                    current.left = TreeNode()
                    current = current.left
            elif char == '1':
                if current.right is None:
                    current.right = TreeNode()
                    current = current.right
            else:
                raise ValueError("Недопустимый символ '" + char + "' в двоичном коде '" +
path + "'")

        # Exception: Перезапись значения
        if current.value is not None:
            raise ValueError(
                "Обнаружена перезапись: путь '" + path + "' уже занят числом '" +
                str(current.value) + "', невозможно записать число '" + str(value) + "'."
            )

        current.value = value

    # Проверка на наличие пустых ячеек
    check_for_empty_nodes(root.left, "0")
    check_for_empty_nodes(root.right, "1")

    return root

def check_for_empty_nodes(node, current_path):
    if node is None:
        return

    # Exception: Если узел существует, но его значение не задано
    if node.value is None:
        raise ValueError(
            "Обнаружена пустая промежуточная ячейка по пути '" + current_path +
```



```

        "'. Дерево не может быть построено."
    )

    # Рекурсивная проверка потомков
    check_for_empty_nodes(node.left, current_path + "0")
    check_for_empty_nodes(node.right, current_path + "1")

def parse_line(line, line_id): # Проверка данных и преобразование строки
    line = line.strip() # Убирает лишние пробелы
    if not line:
        return None

    parts = line.split()
    if len(parts) != 2:
        print("Ошибка в записи " + str(line_id) + ": ожидалось 2 элемента, получено " +
str(len(parts)))
        return None

    try:
        val = int(parts[0])
    except ValueError:
        print("Ошибка в записи " + str(line_id) + ": '" + parts[0] + "' не является целым
числом")
        return None

    path = parts[1]
    return val, path

def read_from_file(): # Чтение из файла
    file_path = input("Введите имя или путь к файлу: ").strip() # Ожидается формат "file.txt"
    data_pairs = []

    try:
        with open(file_path, "r", encoding="utf-8") as file:
            line_num = 0
            for line in file:
                line_num += 1
                parsed = parse_line(line, "в строке " + str(line_num))
                if parsed:
                    data_pairs.append(parsed)
    except FileNotFoundError: # Exception: Ошибки при открытии файла
        print("Ошибка: Файл '" + file_path + "' не найден.")
        return None
    except PermissionError:
        print("Ошибка: Нет прав на чтение файла '" + file_path + "'.")
        return None
    except Exception as e:
        print("Непредвиденная ошибка при чтении файла: " + str(e))
        return None

    return data_pairs

```

```

def read_from_console(): # Ввод с клавиатуры
    print("\nВыбран ручной ввод данных")
    print("Введите данные в формате: [число] [путь] через пробел")
    print("Для завершения ввода введите 0")
    print("-" * 45) # Разделитель

    data_pairs = []
    entry_num = 0

    while True:
        entry_num += 1
        try:
            line = input("Запись " + str(entry_num) + ": ").strip()
            if line == "0":
                print("Ввод завершен.")
                break

            parsed = parse_line(line, entry_num)
            if parsed:
                data_pairs.append(parsed)
        except Exception as e: # Exception
            print("Ошибка при вводе: " + str(e))

    return data_pairs

def main():
    print("Выберите способ ввода данных:")
    print("1: Прочитать данные из файла")
    print("2: Ввести вручную")

    choice = input("Ваш выбор: ").strip()
    data_pairs = []

    if choice == "1":
        data_pairs = read_from_file()
    elif choice == "2":
        data_pairs = read_from_console()
    else:
        print("Неверный выбор. Программа завершена.")
        return

    if not data_pairs:
        print("Не найдены корректные данные для построения. Сборка дерева отменена.")
        return

    try:
        tree_root = build_tree(data_pairs) # Переменная была создана для вывода дерева,
        # функция вывода не была реализована
        print("Дерево успешно построено.")
    except ValueError as val_err:
        print("\nОшибка построения дерева " + str(val_err))
    except Exception as e:

```

```
print("\nНепредвиденная ошибка при построении дерева: " + str(e))

if __name__ == "__main__":
    print("Построение бинарного дерева по данным из файла.")
    main()
```

Ссылка на репозиторий Github

В данном репозитории программа №1 с названием 15.py, программа №2 с названием 19.py. Файлы для тестирования программы №2 с названиями set1-4.txt
https://github.com/chavasse97/python_ikm_sem2